

Sparse Autoencoders

Unpacking features from superposition · video 2:20

This lesson answers one question: if a model stores features in superposition, can we get them back out? A **sparse autoencoder** (SAE) re-expresses each activation as a combination of many more features than dimensions, only a few of them active at a time. On GPT-2's layer 6, 30 of 6,144 features per token rebuild 80% of the variance and keep 93% of the layer's effect on the next-token loss, and many features are readable: “chief”, a speaker name, a family of learned words.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 7).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The sparse autoencoder	▶ 0:08-0:55	3	☐
2. Is the reconstruction good enough?	▶ 0:55-1:26	2	☐
3. Reading features	▶ 1:26-2:09	2	☐

Before you start: Deep Dive episodes 43 (superposition) and 13 (the residual stream). Code: `code/d44_sae` (PyTorch + transformers, about 30 minutes on a CPU).

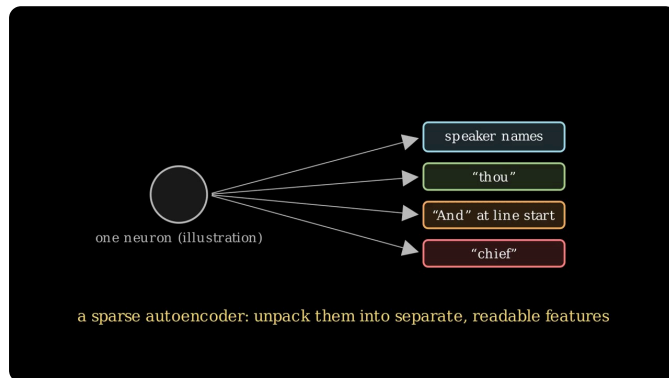
CONCEPT 1 OF 3

The sparse autoencoder

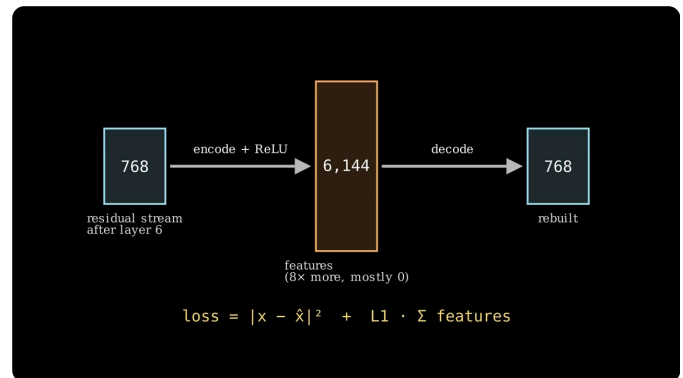
▶ Video 0:08-0:55

Episode 43 showed that sparse features can be stored in overlapping directions, so a single neuron responds to several of them. An SAE tries to undo this. It reads the residual stream after GPT-2's layer 6 (768 numbers per token) and writes it as 6,144 non-negative feature activations, eight times more, then rebuilds the 768 numbers from them:

$$f = \text{ReLU}(W_{\text{enc}} (x - b_{\text{dec}}) + b_{\text{enc}}) \quad \hat{x} = W_{\text{dec}} f + b_{\text{dec}}$$



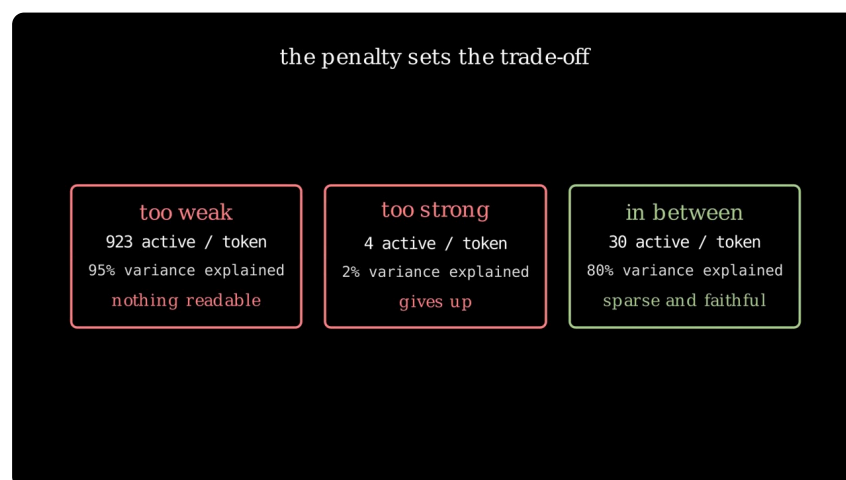
▶ 0:20 Superposition: one neuron responds to several unrelated features (illustration).



▶ 0:38 768 → 6,144 features → 768; the loss rebuilds the input and penalises active features.

The loss has two parts: rebuild x well (squared error), and keep few features on (an L1 penalty on f , weighted by each feature's decoder-column length so the model cannot cheat by shrinking f and growing W_{dec}). The penalty's weight decides the trade-off:

L1 WEIGHT	ACTIVE FEATURES PER TOKEN	VARIANCE EXPLAINED
0.23 (first run)	923	95%
2 (used)	30	80%
8 (800-step sweep)	4	2%



▶ 0:54 The penalty sets the trade-off: 923 active (not readable), 4 active (gives up), 30 active.

KEY IDEA

An SAE trades a little reconstruction accuracy for sparsity: each activation becomes a sum of a few, hopefully meaningful, features.

Check 1: The sparse autoencoder

Answer all 3 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

How many weights do W_{enc} ($6,144 \times 768$) and W_{dec} ($768 \times 6,144$) hold together, plus the biases b_{enc} (6,144) and b_{dec} (768)?

.....

1.2 SHORT ANSWER

Why is a run with 923 active features per token not useful, even though it explains 95% of the variance?

.....

1.3 TRUE OR FALSE

“A stronger L1 penalty always gives better features.”

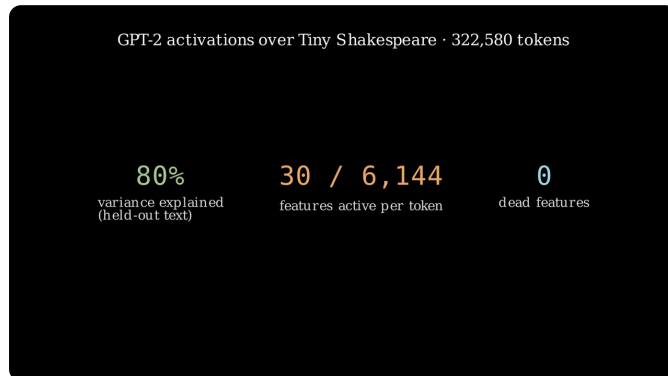
True ☐ False ☐

CONCEPT 2 OF 3

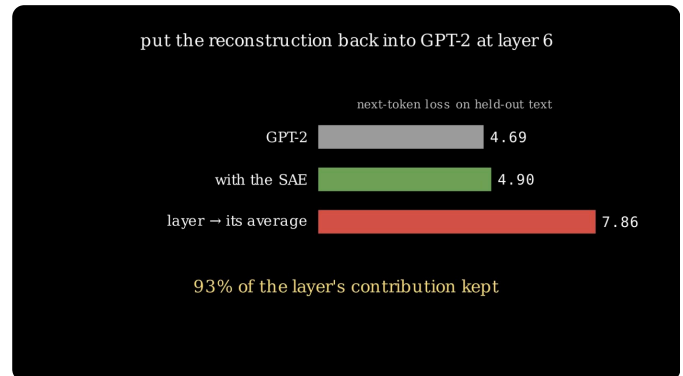
Is the reconstruction good enough?

▶ Video 0:55–1:26

Trained on 322,580 tokens of GPT-2 activations over Tiny Shakespeare, the SAE explains 79.9% of the variance on held-out text, with 30 of 6,144 features active per token on average, and every feature fires somewhere (no **dead features**).



▶ 1:07 Held-out text: 80% variance explained, 30 of 6,144 features active, 0 dead.



▶ 1:25 Spliced into GPT-2: loss 4.69 → 4.90; the layer replaced by its mean: 7.86.

Variance explained can mislead: the missing 20% might be exactly what the model needs. The stronger test is to **splice** the reconstruction into GPT-2 in place of the real activations and let it continue. The next-token loss goes from 4.686 to 4.903. For scale, replacing the layer with its average activation gives 7.861. So the SAE keeps $(7.861 - 4.903) / (7.861 - 4.686) = 93\%$ of what the layer contributes.

KEY IDEA

Judge an SAE by what the model does with its reconstruction, not only by reconstruction error.

Check 2: Is the reconstruction good enough?

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

On average, what fraction of the 6,144 features is active on a token?

2.2 CALCULATION

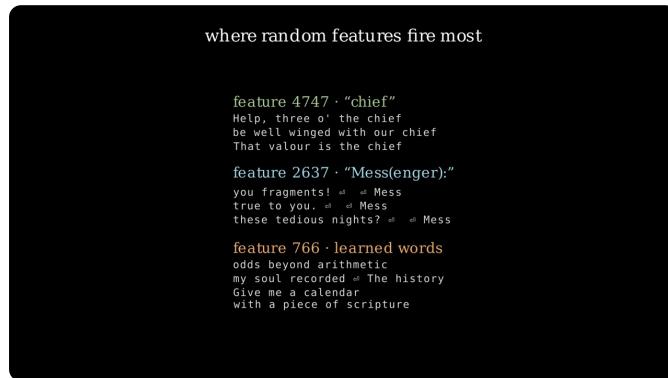
If a different SAE gave a spliced loss of 5.5 (base 4.686, mean-ablated 7.861), what share of the gap would it keep?

CONCEPT 3 OF 3

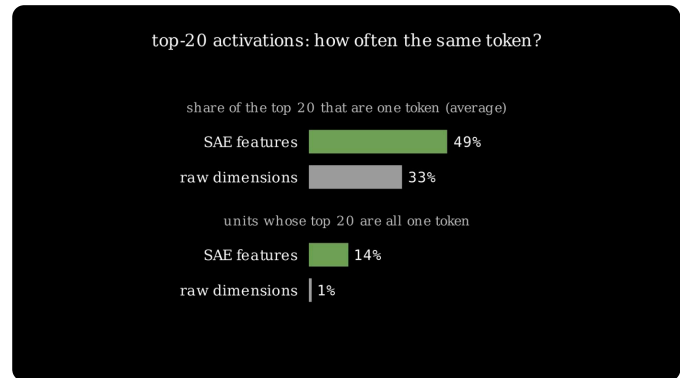
Reading features

▶ Video 1:26–2:09

To read a feature, list the tokens (with context) where it fires most. Among eight random features with moderate frequency: feature 4747 fires on “chief” (“three o’ the chief”, “valour is the chief”); 2637 on “Mess”, the start of the speaker name Messenger; 766 on learned words (arithmetic, history, mechanics, calendar, scripture). Others are less clean, mixing several patterns.

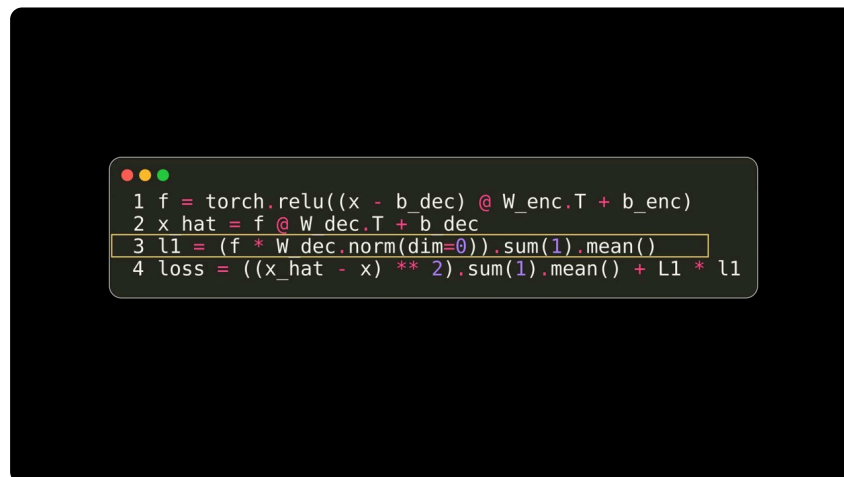


▶ 1:42 Where random features fire most: “chief”, “Mess(enger):”, learned words.



▶ 1:58 Top-20 activations that are one token: SAE features 49% vs raw dimensions 33%; all one token: 14% vs 1%.

A rough comparison with the raw dimensions: for each unit, take its top 20 activations and count how many are the most common token. SAE features: 49% on average, raw dimensions: 33%. Units whose top 20 are all one token: 14% of SAE features, 1% of raw dimensions. Token purity is only one kind of meaning (feature 766 is about a topic, not a token), but it shows features are more specific than dimensions.



▶ 2:08 The SAE in code: encode with a ReLU, decode, L1 weighted by decoder norms.

KEY IDEA

SAE features are far more specific than raw dimensions, which is why they are the main tool for reading what large models represent.

Check 3: Reading features

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 SHORT ANSWER

Feature 766 fires on arithmetic, history, mechanics, calendar, scripture. Why would the token-purity measure underrate it?

3.2 HANDS-ON CODE

Try it yourself. Write the SAE forward pass and loss.

```
import torch
D, M, L1 = 768, 6144, 2.0
W_enc = torch.randn(M, D) / D ** 0.5; W_dec = W_enc.T.clone()
b_enc, b_dec = torch.zeros(M), torch.zeros(D)
x = torch.randn(16, D)
f = torch.relu((x - b_dec) @ W_enc.T + b_enc)
x_hat = f @ W_dec.T + b_dec
loss = ((x_hat - x) ** 2).sum(1).mean() + L1 * (f * W_dec.norm(dim=0)).sum(1).mean()
print(f.shape, (f > 0).float().sum(1).mean(), loss)
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The sparse autoencoder · rewatch ▶ 0:08-0:55

1.1 9,444,096. $2 \times 6,144 \times 768 = 9,437,184$, plus $6,144 + 768 = 6,912$.

1.2 With hundreds of features on at once, no single feature stands for anything you can read: the code is dense again. Sparsity is what makes features interpretable.

1.3 False. Too strong and the SAE gives up: at weight 8 it kept 4 features on and explained 2% of the variance.

2. Is the reconstruction good enough? · rewatch ▶ 0:55-1:26

2.1 About 0.5%. $30 / 6,144 \approx 0.0049$.

2.2 About 74%. $(7.861 - 5.5) / (7.861 - 4.686) = 2.361 / 3.175 \approx 0.74$.

3. Reading features · rewatch ▶ 1:26-2:09

3.1 It responds to a theme across different tokens, so its top 20 are not one token even though it is meaningful. Purity only measures single-token features.

3.2 f has shape (16, 6144); untrained, about half the features are active ($\approx 3,072$). Training with the L1 penalty brings that down to about 30 (step 1 of the episode's run: 3,072).